2.10 Expressions and Assignments (Lab 2.0)

LAB ACTIVITY | 2.10.1: Expressions and Assignments (Lab 2.0) | Full screen | 0 / 20

Follow the pattern from the lab in unit 1 for this set of lab exercises. Create a new directory, `lab02` for these exercises; download/rename/edit template.py as specified; and use Thonny. This unit will have our first content quiz, so we suggest that you do at least this exercise before doing the quiz.

Exercise

Create a module called `expressions.py` (using the template listed above). Do these exercises in Thonny's Python shell and include your notes as comments in the `expressions.py` submission module.

1. Be sure that you can explain the value of each of the following expressions. Evaluate these expressions using the Thonny Python shell, but include comments in your `expressions.py` solution file to describe any expressions that have surprising results. Feel free to experiment with other expressions.

   ```
   3 + 4
   2 + 3 * 5
   8 - 4 - 2
   (3 + 7) * 2
   13 % 4
   8.2 // 4
   2**10
   5.1 % 2
   ```

   After experimenting with these expressions in a Python shell, document what you found in your `expressions.py` file. For example, your entry for the first expression might say:

   ```
   # 3 + 4 computes the sum of 3 and 4 as expected.
   ```

2. Write a statement that assigns a Python variable name of your choice to an integer value read from the user using a prompt "Please enter an integer: ". Be sure to use a legal Python name: for example is `my_integer` legal? How about `4_an_integer`? Include the statement in your `expressions.py` submission module.

These two exercises will not be carefully checked by the ZyLabs auto-grader. To prepare for the quiz and lab, understand your work; to get credit from the ZyLabs grader, just submit your `expressions.py` file.

Grading

Grading is just like in Lab 1; this section is repeated here for reference:

- **90% — Correctness**: your submitted programs should run properly, according to the instructions given in the assignment and typically (but not always) tested automatically by ZyLabs
- **10% — Understandability** (assessed manually): Someone reading your programs should be able to clearly see who wrote them and how they are organized. For example, the logical blocks of your programs should be described by useful comments and separated by white space.

Note: Labs are graded out of 100 points. The grade you'll see in ZyLabs will be missing points for elements that cannot be auto-graded. Those points will be added manually when the graders update your final lab grade in the Moodle gradebook.

Open new tab | Dock

Run History Tutorial

```
"""CS 108 - Lab/Homework X.X

Describe the module here. Fix the lab number above and the name/date below.

@author: your name(s) here...
@date: semester, year
"""

# Put your solution code here.

```

DESKTOP CONSOLE

Submit for grading

Coding trail of your work What is this?

```
History of your effort will appear here once you begin
working on this zyLab.
```

Latest submission

No submissions yet

Template, Model Solution, and Test Bench Show

Lab statistics Show

Student behavior insights (beta) Show

Similarity detection Show

Student activity Show